25 MIDDLEWARE — THE STAGE BEFORE EVERYTHING ELSE

Middleware is a function that runs on every matching request BEFORE Nest's own pipeline (guards, pipes, handler) even starts. It comes straight from Express — Nest doesn't reinvent it, it just gives you a class-based way to register the same thing you'd normally pass to `app.use()`.

```
Client Request
     |
MIDDLEWARE   <-- runs here, first. Has raw (req, res, next).
     |
Guards           (see theory/17-guards.txt)
     |
Interceptors (before)
     |
Pipes            (see theory/09-pipes.txt)
     |
Route Handler
     |
Interceptors (after)
     |
Response
```

This project has NO custom middleware yet — auth.guard.ts and roles.guard.ts are guards, not middleware (see theory/26 for exactly why that choice was made). Everything below is a designed example to explain the mechanism.

THE NestMiddleware INTERFACE

A class-based middleware implements NestMiddleware, with one method: use(req, res, next).

```ts
import { Injectable, NestMiddleware } from '@nestjs/common';
import { Request, Response, NextFunction } from 'express';

@Injectable()
export class LoggerMiddleware implements NestMiddleware {
  use(req: Request, res: Response, next: NextFunction) {
    console.log(`${req.method} ${req.originalUrl}`);
    next(); // MUST call this, or the request hangs forever
  }
}
```

req, res	The raw Express request/response objects — no Nest wrapping, no ExecutionContext. This is the one stage where you're working with Express directly.
next()	Hands control to the next thing in the chain (another middleware, then guards, then the rest of the pipeline). Forgetting to call it is the #1 middleware bug — the client just hangs with no response and no error.

You can also write it as a plain function instead of a class (FUNCTIONAL MIDDLEWARE) when it needs no dependency injection:

```ts
export function logger(req: Request, res: Response, next: NextFunction) {
  console.log(`${req.method} ${req.originalUrl}`);
  next();
}
```

HOW IT'S WIRED UP — configure() + MiddlewareConsumer

Unlike guards/pipes, middleware is NOT attached with a decorator on the controller or handler. It's registered in a module via the `configure` method, which only exists if the module implements NestModule:

```ts
// app.module.ts
import { Module, NestModule, MiddlewareConsumer } from '@nestjs/common';
import { LoggerMiddleware } from './common/middleware/logger.middleware';

@Module({ /* imports, controllers, providers */ })
export class AppModule implements NestModule {
  configure(consumer: MiddlewareConsumer) {
    consumer
      .apply(LoggerMiddleware)
      .forRoutes('*'); // every route in the app
  }
}
```

`forRoutes()` accepts a lot of shapes, letting you scope middleware far more precisely than "everywhere":

```ts
.forRoutes('task')                          // whole TaskController
.forRoutes({ path: 'task', method: RequestMethod.POST }) // one verb
.forRoutes(TaskController)                  // by class reference
```

You can also EXCLUDE specific routes:

```ts
consumer
  .apply(LoggerMiddleware)
  .exclude({ path: 'task', method: RequestMethod.GET })
  .forRoutes(TaskController);
```

And chain multiple middleware in one `.apply()` call — they run in the order listed:

```ts
consumer.apply(cors(), helmet(), LoggerMiddleware).forRoutes('*');
```

WHAT MIDDLEWARE IS ACTUALLY GOOD FOR

Because it runs before Nest's own concepts even exist for the request, middleware is the right tool for concerns that are truly request/response-level plumbing, not "business" authorization or validation:

- Logging every request (method, url, timing)
- CORS headers (`cors()`)
- Security headers (`helmet()`)
- Body parsing (Express already does this by default via `app.use(express.json())`, wired in automatically by Nest)

- Rewriting/attaching things onto the raw `req` object itself before ANYTHING else — e.g. a correlation/request ID: req['requestId'] = crypto.randomUUID();

It is the WRONG tool for "is this user allowed?" (that's a guard, theory/17-guards.txt) or "is this DTO field valid?" (that's a pipe, theory/09-pipes.txt) — middleware has no concept of the route handler, its metadata, or its typed arguments. It only sees the raw HTTP req/res.

GLOBAL MIDDLEWARE — THE OTHER WAY, IN main.ts

For middleware that truly applies to the whole app with no per-route nuance, you can skip `configure()` entirely and register it where the app is created:

```
// main.ts
async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.use(helmet());
  app.use(cors());
  await app.listen(3000);
}
```

This is exactly parallel to `app.useGlobalGuards()` / `useGlobalPipes()` (theory/11-builtin-vs-custom-pipes.txt) — same tradeoff: quick and global, but no DI, so only reach for it when the middleware needs no injected dependencies.

★ RULE OF THUMB

Reach for middleware when the concern is about the RAW REQUEST/RESPONSE itself and has no idea what a "controller" or "DTO" even is — logging, CORS, security headers, correlation IDs. The moment the concern is "who is this user" or "is this argument well-formed," you've moved into guard/pipe territory, where Nest gives you ExecutionContext and typed, validated arguments instead of a bare Express req/res.

See also: theory/17-guards.txt, theory/09-pipes.txt, theory/26-middleware-vs-guards.txt, theory/05-express-vs-nestjs.txt